

FC7xxx FLS User Manual

Rev.0.6

Revision History

Revision	Date	Changes
0.1	2023/07/14	Initial release for MCAL V0.1.0
0.3	2023/10/20	Updated for MCAL V0.3.0
0.6	2023/03/29	Updated for MCAL V0.6.0 - Added support for FC7240

Table of Contents

Revision History	2
Table of Contents	3
Chapter 1 FLS Introduction	5
1.1 Requirement Tracing	5
1.2 Hardware Summary	5
Chapter 2 Software Design	6
2.1 Rejected Requirements	6
2.2 File Structure	10
2.3 Macros	10
2.3.1 Macros in Fls.h	10
2.3.2 Macros in Fls_version.h	12
2.3.3 Macros in FLS_Cfg.h	12
2.4 Enums	14
2.4.1 Enumerations in Fls_Types.h	14
2.5 Typedefs	15
2.5.1 Typedefs in Fls_Types.h	15
2.6 Structures	15
2.6.1 Fls_ConfigType	15
2.7 API Functions	16
2.7.1 Functions in Fls.h	16
2.8 Hardware Functions	19
2.8.1 Functions in Fls_Flash.h	19
2.9 API Sequence Diagram	21
2.9.1 Initialization	21
2.9.2 Synchronous Functions	22
2.9.3 Asynchronous Functions	23
2.9.4 Canceling a Running Job	24
Chapter 3 Tresos Configuration Items	25
3.1 Container Inclusion Relation	25
3.2 Containers and Variables	26
3.2.1 IMPLEMENTATION_CONFIG_VARIANT	26
3.2.2 FlsGeneral	26
3.2.3 FlsConfigSet	31
3.2.4 FlsSectorList	34
3.2.5 FlsPublishedInformation	35

3.2.6	CommonPublishedInformation	38
Chapter 4 Configuration Guides.....		41
4.1	Configuration Item Constraint.....	41
4.2	FLS Usage Common Steps	42
4.3	Demo Configuration (FC7300F8M).....	42

Chapter 1 FLS Introduction

1.1 Requirement Tracing

The design of this module follows the specifications of the Flash driver specified in AUTOSAR Classic Platform Release 4.6.0. For detailed requirements, refer to the *AUTOSAR_SWS_FlashDriver*.

1.2 Hardware Summary

The flash driver provides services for reading, writing and erasing flash memory and a configuration interface for setting / resetting the write / erase protection if supported by the underlying hardware.

In application mode of the ECU, the flash driver is only to be used by the Flash EEPROM emulation module for writing data. It is not intended to write program code to flash memory in application mode. This shall be done in boot mode which is out of scope of AUTOSAR.

Chapter 2 Software Design

2.1 Rejected Requirements

Rejected Requirement 1 SWS_Fls_00088	
Description	The functional requirements and the functional scope are the same for both internal and external drivers. Hence the API is semantically identical.
Rejection Reason	Disable config external drivers, only use internal flash.

Rejected Requirement 2 SWS_Fls_00107	
Description	file structure.
Rejection Reason	Internal flash does not use interrupt, so it does not include Flash_Irq.c file.

Rejected Requirement 3 SWS_Fls_00319	
Description	The transient fault code FLS_E_UNEXPECTED_FLASH_ID shall be reported when the expected flash ID is not matched.
Rejection Reason	FLS_E_UNEXPECTED_FLASH_ID is applicable only for external Fls driver.h, here only support internal flash, so not used.

Rejected Requirement 4 SWS_Fls_00144	
Description	During the initialization of the external flash driver, the FLS module shall check the hardware ID of the external flash device against the corresponding published parameter. If a hardware ID mismatch occurs, the FLS module shall report the error code FLS_E_UNEXPECTED_FLASH_ID to the Default Error Tracer (DET), set the FLS module status to FLS_E_UNINIT and shall not initialize itself.
Rejection Reason	Only support internal flash, so not check and report.

Rejected Requirement 5 SWS_Fls_00137	
Description	The FLS module's implementer shall place the code of the flash access routines into a separate C-module Fls_ac.c.
Rejection Reason	Flash access code is fixed in ROM, and just call the corresponding API.

Rejected Requirement 6 SWS_Fls_00213	
Description	The FLS module's main processing routine shall access the flash access code routines by means of the respective function pointer contained in the FLS module's configuration set (post-compile parameters) regardless whether the flash access code routines have been loaded to RAM or whether they can be executed directly from (flash) ROM.
Rejection Reason	Flash access code is fixed in ROM and just call the corresponding API.

Rejected Requirement 7 SWS_Fls_0048	
--	--

Description	If supported by hardware, the function Fls_Init shall set the flash memory erase/write protection as provided in the configuration set.
Rejection Reason	not implemented

Rejected Requirement 8 SWS_Fls_00221

Description	The job of the function Fls_Erase shall erase a flash memory block starting from FlsBaseAddress + TargetAddress of size Length. Note: Length will be rounded up to the next full sector boundary since only complete flash sectors can be erased.
Rejection Reason	FlsBaseAddress not used, unclear purpose.

Rejected Requirement 9 SWS_Fls_00020

Description	If development error detection for the module Fls is enabled: the function Fls_Erase shall check that the erase start address (FlsBaseAddress + TargetAddress) is aligned to a flash sector boundary and that it lies within the specified lower and upper flash address boundaries. If this check fails, the function Fls_Erase shall reject the erase request, raise the development error FLS_E_PARAM_ADDRESS and return with E_NOT_OK.
Rejection Reason	FlsBaseAddress not used, unclear purpose.

Rejected Requirement 10 SWS_Fls_00145

Description	If possible, e.g. with interrupt controlled implementations, the FLS module shall start the first round of the erase job directly within the function Fls_Erase to reduce overall runtime.
Rejection Reason	not use interrupt, so not do this

Rejected Requirement 11 SWS_Fls_00226

Description	The job of the function Fls_Write shall program a flash memory block with data provided via SourceAddressPtr starting from FlsBaseAddress + TargetAddress of size Length.
Rejection Reason	FlsBaseAddress not used, unclear purpose.

Rejected Requirement 12 SWS_Fls_00026

Description	If development error detection for the module Fls is enabled: the function Fls_Write shall check that the write start address (FlsBaseAddress + TargetAddress) is aligned to a flash page boundary and that it lies within the specified lower and upper flash address boundaries. If this check fails, the function Fls_Write shall reject the write request, raise the development error FLS_E_PARAM_ADDRESS and return with E_NOT_OK.
Rejection Reason	FlsBaseAddress not used, unclear purpose.

Rejected Requirement 13 SWS_Fls_00239

Description	The read job of the function Fls_Read shall copy a continuous flash memory block starting from FlsBaseAddress + SourceAddress of size Length to the buffer pointed to by TargetAddressPtr.
Rejection Reason	FlsBaseAddress not used, unclear purpose.

Rejected Requirement 14 SWS_Fls_00097

Description	If development error detection for the module Fls is enabled: the function Fls_Read shall check that the read start address (FlsBaseAddress + SourceAddress) lies within the specified lower and upper flash address boundaries. If this check fails, the function Fls_Read shall reject the read job, raise development error FLS_E_PARAM_ADDRESS and return with E_NOT_OK.
Rejection Reason	FlsBaseAddress not used, unclear purpose.

Rejected Requirement 15 SWS_Fls_00244

Description	The job of the function Fls_Compare shall compare a continuous flash memory block starting from FlsBaseAddress + SourceAddress of size Length with the buffer pointed to by TargetAddressPtr.
Rejection Reason	FlsBaseAddress not used, unclear purpose.

Rejected Requirement 16 SWS_Fls_00150

Description	If development error detection for the module Fls is enabled: the function Fls_Compare shall check that the compare start address (FlsBaseAddress + SourceAddress) lies within the specified lower and upper flash address boundaries. If this check fails, the function Fls_Compare shall reject the compare job, raise the development error FLS_E_PARAM_ADDRESS and return with E_NOT_OK.
Rejection Reason	FlsBaseAddress not used, unclear purpose.

Rejected Requirement 17 SWS_Fls_00379

Description	The verification job of the function Fls_BlankCheck shall check, that the continuous flash memory area starting from FlsBaseAddress + TargetAddress of size Length is erased.
Rejection Reason	FlsBaseAddress not used, unclear purpose.

Rejected Requirement 18 SWS_Fls_00380

Description	If development error detection for the module FLS is enabled; the function Fls_BlankCheck shall check that the verification start address (FlsBaseAddress + TargetAddress) lies within the specified lower and upper flash address boundaries. If this check fails, the function Fls_BlankCheck shall reject the verification job, raise the development error FLS_E_PARAM_ADDRESS and return with E_NOT_OK.
Rejection Reason	FlsBaseAddress not used, unclear purpose.

Rejected Requirement 19 SWS_Fls_00193

Description	Depending on implementation, callback routines provided and/or invoked by the FLS module may be called on interrupt level. The module providing those routines has therefore to make sure that their runtime is reasonably short, i.e. since callbacks may be propagated upward through several software layers.
--------------------	--

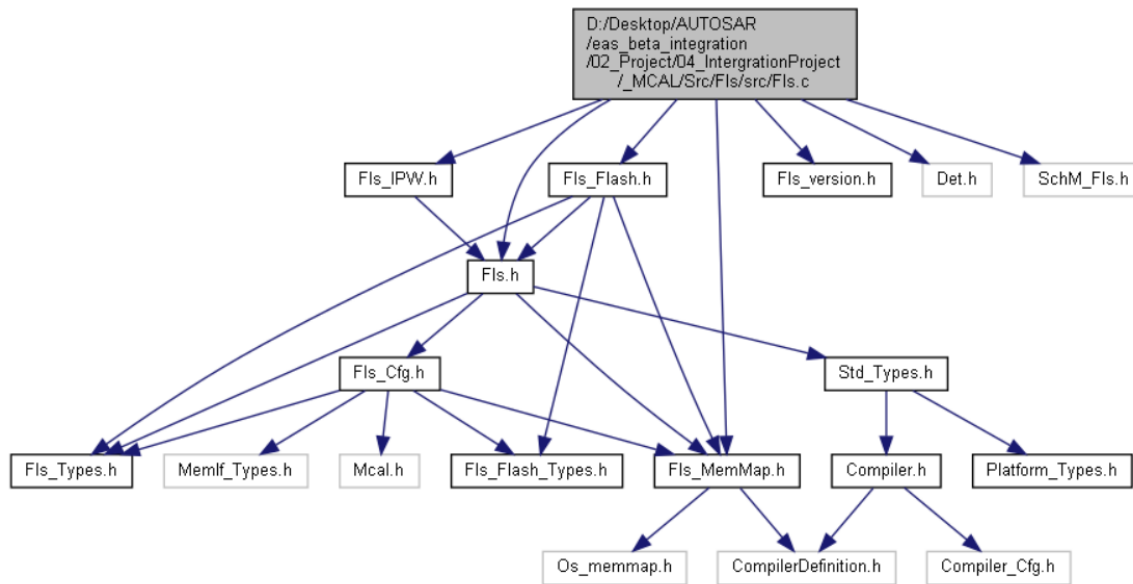
Rejection Reason	Not necessary and not implemented.
-------------------------	------------------------------------

Rejected Requirement 20 SWS_Fls_00233	
Description	The FLS module's implementer shall locate the interrupt service routine in Fls_Irq.c.
Rejection Reason	Hardware does not support Interrupt, so not have FLS_Irq.c

Rejected Requirement 21 SWS_Fls_00354	
Description	The type declaration for Fls_ConfigType shall be located in the file Fls.h.
Rejection Reason	Fls_ConfigType is included into Fls.h from Fls_Cfg.h.

Rejected Requirement 22 ECUC_Fls_00317				
Description	SWS Item	ECUC_Fls_00317		
	Name	FlsSpiReference		
	Parent Container	FlsExternalDriver		
	Description	Reference to SPI sequence (required for external Flash drivers).		
	Multiplicity	1..*		
	Type	Symbolic name reference to [SpiSequence]		
	Post-Build Variant Multiplicity	false		
	Post-Build Variant Value	false		
	Multiplicity Configuration Class	Pre-compile time	X	All Variants
		Link time	--	
		Post-build time	--	
	Value Configuration Class	Pre-compile time	X	All Variants
		Link time	--	
		Post-build time	--	
	Scope / Dependency	scope: local		
Rejection Reason	Not used, so not implemented.			

2.2 File Structure



2.3 Macros

2.3.1 Macros in Fls.h

- #define **FLS_E_PARAM_CONFIG** 0x01U
- #define **FLS_E_PARAM_ADDRESS** 0x02U
- #define **FLS_E_PARAM_LENGTH** 0x03U
- #define **FLS_E_PARAM_DATA** 0x04U
API service called with wrong parameter.
- #define **FLS_E_UNINIT** 0x05U
API service called without module initialization.
- #define **FLS_E_BUSY** 0x06U
API service called while driver was still busy.
- #define **FLS_E_ERASE_FAILED** 0x01U
Flash erase failed (HW).
- #define **FLS_E_WRITE_FAILED** 0x02U
Flash write failed (HW).
- #define **FLS_E_READ_FAILED** 0x03U
Flash read failed (HW).

- **#define FLS_E_COMPARE_FAILED 0x04U**
Flash compare failed (HW).
- **#define FLS_E_UNEXPECTED_FLASH_ID 0x05U**
Expected hardware ID not matched.
- **#define FLS_E_VERIFY_ERASE_FAILED 0x07U**
Erase verification (blank check) failed.
- **#define FLS_E_VERIFY_WRITE_FAILED 0x08U**
Write verification (compare) failed.
- **#define FLS_E_TIMEOUT 0x09U**
Timeout exceeded.
- **#define FLS_E_PARAM_POINTER 0x0AU**
API service called with NULL pointer.
- **#define FLS_INIT_ID 0x00U**
Init service ID.
- **#define FLS_ERASE_ID 0x01U**
Erase service ID.
- **#define FLS_WRITE_ID 0x02U**
Write service ID.
- **#define FLS_CANCEL_ID 0x03U**
Cancel service ID.
- **#define FLS_GETSTATUS_ID 0x04U**
GetStatus service ID.
- **#define FLS_GETJOBRESULT_ID 0x05U**
GetJobResult service ID.
- **#define FLS_MAINFUNCTION_ID 0x06U**
MainFunction service ID.
- **#define FLS_READ_ID 0x07U**
Read service ID.
- **#define FLS_COMPARE_ID 0x08U**
Compare service ID.

- #define **FLS_SETMODE_ID** 0x09U
SetMode service ID.
- #define **FLS_GETVERSIONINFO_ID** 0x10U
GetVersionInfo service ID.
- #define **FLS_BLANK_CHECK_ID** 0x11U
BlankCheck service ID.

2.3.2 Macros in Fls_version.h

- #define **FLS_AR_RELEASE_MAJOR_VERSION** 4
- #define **FLS_AR_RELEASE_MINOR_VERSION** 6
- #define **FLS_AR_RELEASE_REVISION_VERSION** 0
- #define **FLS_MODULE_ID** 92
- #define **FLS_SW_MAJOR_VERSION** 0
- #define **FLS_SW_MINOR_VERSION** 6
- #define **FLS_SW_PATCH_VERSION** 0
- #define **FLS_VENDOR_ID** 174

2.3.3 Macros in FLS_Cfg.h

- #define **FLS_CFG_AR_RELEASE_MAJOR_VERSION** 4
- #define **FLS_CFG_AR_RELEASE_MINOR_VERSION** 6
- #define **FLS_CFG_AR_RELEASE_REVISION_VERSION** 0
- #define **FLS_CFG_SW_MAJOR_VERSION** 0
- #define **FLS_CFG_SW_MINOR_VERSION** 6
- #define **FLS_CFG_SW_PATCH_VERSION** 0
- #define **FLS_CFG_VENDOR_ID** 174

- #define **FLS_AC_LOAD_ON_JOB_START** (STD_OFF)
Flash access code loaded on job start / unloaded on job end or error.
- #define **FLS_CANCEL_API** (STD_ON)
Compile switch to enable and disable the Fls_Cancel function.
- #define **FLS_COMPARE_API** (STD_ON)
Compile switch to enable and disable the Fls_Compare function.
- #define **FLS_BLANK_CHECK_API** (STD_ON)
Compile switch to enable and disable the Fls_BlankCheck function.
- #define **FLS_DEV_ERROR_DETECT** (STD_ON)
Pre-processor switch to enable and disable development error detection.
- #define **FLS_GET_JOB_RESULT_API** (STD_ON)
Compile switch to enable and disable the Fls_GetJobResult function.
- #define **FLS_GET_STATUS_API** (STD_ON)
Compile switch to enable and disable the Fls_GetStatus function.
- #define **FLS_SET_MODE_API** (STD_ON)
Compile switch to enable and disable the Fls_SetMode function.
- #define **FLS_USE_INTERRUPTS** (STD_OFF)
Pre-processor switch to enable / disable the API to use interrupts for erase and write jobs.
- #define **FLS_VERSION_INFO_API** (STD_ON)
Pre-processor switch to enable / disable the API to read out the modules version information.
- #define **FLS_WRITE_VERIFY_CHECK** (STD_ON)
Pre-processor switch to enable / disable the write verify check.
- #define **FLS_ERASE_VERIFY_CHECK** (STD_ON)
Pre-processor switch to enable / disable the erase blank check.
- #define **FLS_TIMEOUT_HANDLING** (STD_ON)
Timeout handling enabled.
- #define **FLS_AC_SIZE_ERASE** (0U)
Number of bytes in RAM needed for the erase flash access code.

- #define **FLS_AC_SIZE_WRITE** (0U)
Number of bytes in RAM needed for the write flash access code.
- #define **FLS_ERASED_VALUE** (0xffffffffU)
The contents of an erased flash memory cell.
- #define **FLS_DRIVER_INDEX** (0U)
Fls Driver Index.
- #define **D_FLASH_BASE_ADDR** (0x4000000U)
The flash memory start address.
- #define **D_FLASH_PAGE_SIZE** (0x10U)
Size of one page of this sector.
- #define **D_FLASH_SECTOR_SIZE** (0x2000U)
Size of one sector.
- #define **D_FLASH_MAX_WRITE_SIZE** (0x80U)
Size of once max write.

2.4 Enums

2.4.1 Enumerations in Fls_Types.h

2.4.1.1 Fls_JobType

Enumeration	Fls_JobType	
Description	Type of job currently executed by Fls_MainFunction.	
Values	Value	Description
	FLS_JOB_ERASE	Erase one or more complete flash sectors.
	FLS_JOB_WRITE	Write one or more complete flash pages.
	FLS_JOB_READ	Read one or more bytes from flash memory.
	FLS_JOB_COMPAR	Compare data buffer with content of flash memory.
	FLS_JOB_BLANK_CHECK	Check content of erased flash memory area.

2.4.1.2 Fls_LLDJobType

Enumeration	Fls_LLDJobType	
Description	Type of job currently executed by Fls_LLDMainFunction.	
Values	Value	Description
	FLASH_JOB_NONE	No job executed by Fls_LLDMainFunction.
	FLASH_JOB_ERASE	Erase one flash sector.
	FLASH_JOB_ERASE_TEMP	Complete erase and start an interleaved erase flash sector.
	FLASH_JOB_WRITE	Write one or more complete flash pages.

	FLASH_JOB_ERASE_BLANK_CHECK	Erase blank check of flash sector.
--	-----------------------------	------------------------------------

2.4.1.3 Fls_LLDReturnTypes

Enumeration	Fls_LLDReturnTypes	
Description	Result of low-level flash operation.	
Values	Value	Description
	FLASH_E_OK	Operation succeeded.
	FLASH_E_FAILED	Operation failed due to hardware error.
	FLASH_E_BLOCK_INCONSISTENT	Data buffer doesn't match with content of flash memory.
	FLASH_E_PENDING	Operation is pending.
	FLASH_E_PARTITION_ERR	FlexNVM partition ratio error.

2.5 Typedefs

2.5.1 Typedefs in Fls_Types.h

- typedef **uint32 Fls_SectorIndexType**
Logical sector index.
- typedef **uint16 Fls_CrcType**
CRC computed over config set.
- typedef **uint32 Fls_AddressType**
Address offset from the configured flash base address to access a certain flash memory area.
- typedef **uint32 Fls_LengthType**
Number of bytes to read, write, erase, compare.
- typedef **uint32 Fls_SectorCountType**
Number of configured sectors.
- typedef **void (*Fls_JobEndNotificationPtrType)(void);**
Pointer type of Fls_JobEndNotification function.
- typedef **void (*Fls_JobErrorNotificationPtrType)(void);**
Pointer type of Fls_JobErrorNotification function.

2.6 Structures

2.6.1 Fls_ConfigType

Structure	Fls_ConfigType
Description	A pointer to such a structure is provided to the flash driver initialization routine for configuration of the driver and flash memory hardware.
Diagram	N/A

Data Fields	- Fls_JobEndNotificationPtrType jobEndNotificationPtr Pointer to job end notification function. - Fls_JobErrorNotificationPtrType jobErrorNotificationPtr Pointer to job error notification function. - MemIf_ModeType eDefaultMode Default FLS device mode after initialization (MEMIF_MODE_FAST, MEMIF_MODE_SLOW). - Fls_LengthType u32MaxReadFastMode Max number of bytes to read in one cycle of Fls_MainFunction (fast mode). - Fls_LengthType u32MaxReadNormalMode Max number of bytes to read in one cycle of Fls_MainFunction (normal mode). - Fls_LengthType u32MaxWriteFastMode Max number of bytes to write in one cycle of Fls_MainFunction (fast mode). - Fls_LengthType u32MaxWriteNormalMode Max number of bytes to write in one cycle of Fls_MainFunction (normal mode). - Fls_SectorCountType u32BlockCount Number of configured logical sectors. - const Fls_AddressType (*paBlockEndAddr)[] Pointer to array containing last logical address of each configured sector. - const uint16 (*paNumberOfSectorCross)[] Pointer to array containing cross number of HW sector. - const Fls_AddressType (*paBlockStartAddr)[] Pointer to array containing start logical address of each configured sector. - const Fls_Flash_InternalSectorInfoType * const (*pBlockList)[] Pointer to array containing physical sector ID of each configured sector. - Fls_CrcType u16ConfigCrc FLS Config Set CRC checksum.
--------------------	---

2.7 API Functions

2.7.1 Functions in Fls.h

2.7.1.1 void Fls_Init(const Fls_ConfigType * pConfigPtr)

Function	void Fls_Init(const Fls_ConfigType * pConfigPtr)	
Description	The function initializes Fls module.	
Diagram	N/A	
Parameters	Parameter	Description
	pConfigPtr	Pointer to flash driver configuration set.
Returns	None	

2.7.1.2 Std_ReturnType Fls_Erase(Fls_AddressType u32TargetAddress, Fls_LengthType u32Length)

Function	Std_ReturnType Fls_Erase(Fls_AddressType u32TargetAddress, Fls_LengthType u32Length)	
Description	Erases flash sector(s).	
Diagram	N/A	
Parameters	Parameter	Description

	u32TargetAddress	Target address in flash memory.
	u32Length	Number of bytes to erase.
Returns	Std_ReturnType:	

2.7.1.3 Std_ReturnType Fls_Write(Fls_AddressType u32TargetAddress, const uint8* pSourceAddressPtr, Fls_LengthType u32Length)

Function	Std_ReturnType Fls_Write(Fls_AddressType u32TargetAddress, const uint8* pSourceAddressPtr, Fls_LengthType u32Length)	
Description	Writes one or more complete flash pages.	
Diagram	NA	
Parameters	Parameter	Description
	TargetAddress	Target address in flash memory. This address offset will be added to the flash memory base address.
	SourceAddressPtr	Pointer to source data buffer.
	Length	Number of bytes to write.
Returns	Std_ReturnType	

2.7.1.4 void Fls_Cancel(oid)

Function	void Fls_Cancel(oid)
Description	Cancels an ongoing job.
Diagram	N/A
Parameters	None
Returns	None

2.7.1.5 MemIf_StatusType Fls_GetStatus(void)

Function	MemIf_StatusType Fls_GetStatus(void)
Description	Returns the driver state.
Diagram	N/A
Parameters	None
Returns	MemIf_StatusType

2.7.1.6 MemIf_JobResultType Fls_GetJobResult (void)

Function	MemIf_JobResultType Fls_GetJobResult (void)
Description	Returns the result of the last job.
Diagram	N/A
Parameters	None
Returns	MemIf_JobResultType

2.7.1.7 Std_ReturnType Fls_Read(Fls_AddressType u32SourceAddress, uint8* pTargetAddressPtr, Fls_LengthType u32Length)

Function	Std_ReturnType Fls_Read(Fls_AddressType u32SourceAddress, uint8* pTargetAddressPtr, Fls_LengthType u32Length)	
Description	Reads from flash memory.	
Diagram	N/A	
Parameters	Parameter	Description
	u32SourceAddress	Source address in flash memory. This address offset will be added to the flash memory base address.
	u32Length	Number of bytes to read.
Returns	Std_ReturnType	

2.7.1.8 Std_ReturnType Fls_Compare(Fls_AddressType u32SourceAddress, const uint8* pTargetAddressPtr, Fls_LengthType u32Length)

Function	Std_ReturnType Fls_Compare(Fls_AddressType u32SourceAddress, const uint8* pTargetAddressPtr, Fls_LengthType u32Length)	
Description	Compares the contents of an area of flash memory with that of an application data	
Diagram	N/A	
Parameters	Parameter	Description
	u32SourceAddress	Source address in flash memory. This address offset will be added to the flash memory base address.
	u32Length	Number of bytes to compare.
Returns	N/A	

2.7.1.9 void Fls_SetMode(MemIf_ModeType eMode)

Function	void Fls_SetMode(MemIf_ModeType eMode)	
Description	Sets the flash driver's operation mode.	
Diagram	N/A	
Parameters	Parameter	Description
	MEMIF_MODE_SLOW	Slow read access.
	MEMIF_MODE_FAST	Fast read access.
Returns	N/A	

2.7.1.10 void Fls_GetVersionInfo(Std_VersionInfoType* pVersioninfoPtr)

Function	void Fls_GetVersionInfo(Std_VersionInfoType* pVersioninfoPtr)	
Description	Returns the version information of this module.	
Diagram	N/A	
Parameters	Parameter	Description
	pVersioninfoPtr	Pointer to where to store the version information of this module.
Returns	None	

2.7.1.11 Std_ReturnType Fls_BlankCheck(Fls_AddressType u32TargetAddress, Fls_LengthType u32Length)

Function	Std_ReturnType Fls_BlankCheck(Fls_AddressType u32TargetAddress, Fls_LengthType u32Length)	
Description	The function Fls_BlankCheck shall verify, whether a given memory area has been erased but not (yet) programmed. The function shall limit the maximum number of checked flash cells per main function cycle to the configured value FlsMaxReadNormalMode or FlsMaxReadFastMode respectively.	
Diagram	N/A	
Parameters	Parameter	Description
	u32TargetAddress	Address in flash memory from which the blank check should be started.
	u32Length	Number of bytes to be checked for erase pattern.
Returns	Std_ReturnType	

2.7.1.12 void Fls_MainFunction(void)

Function	void Fls_MainFunction(void)
Description	Performs the processing of jobs.
Diagram	N/A
Parameters	N/A
Returns	N/A

2.8 Hardware Functions

2.8.1 Functions in Fls_Flash.h

2.8.1.1 void Fls_Flash_Init(void)

Function	void Fls_Flash_Init(void)
Description	Initialize flash hardware.
Parameters	N/A
Returns	N/A
Referenced By	Fls_Init

2.8.1.2 void Fls_Flash_MainFunctionErase(void)

Function	void Fls_Flash_MainFunctionErase(void)
Description	Process ongoing erase hardware job.
Parameters	N/A
Returns	N/A
Referenced By	Fls_IPW_LLDMainFunction

2.8.1.3 void Fls_Flash_MainFunctionWrite(void)

Function	void Fls_Flash_MainFunctionWrite(void)
Description	Enable an interrupt.
Parameters	N/A
None	N/A
Referenced By	Fls_IPW_LLDMainFunction

2.8.1.4 Fls_Flash_MainFunctionEraseBlankCheck

Function	Fls_Flash_MainFunctionEraseBlankCheck(void)
Description	Check Erase blank.
Parameters	N/A
Returns	N/A
Referenced By	Fls_IPW_LLDMainFunction

2.8.1.5 void Fls_Flash_Cancel(void)

Function	void Fls_Flash_Cancel(void)
Description	Cancel ongoing erase or write hardware job.
Parameters	N/A
Returns	N/A
Referenced By	Fls_IPW_AbortSuspended

2.8.1.6 Fls_LLDReturnType Fls_Flash_Erase(const Fls_PhysicalSectorType ePhySector)

Function	Fls_LLDReturnType Fls_Flash_Erase(const Fls_PhysicalSectorType ePhySector)	
Description	Erase one complete flash sector.	
Parameters	Parameter	Description
	ePhySector	Physical sector ID.
Returns	Fls_LLDReturnType	
Referenced By	Fls_IPW_Erase	

2.8.1.7 Fls_LLDReturnType Fls_Flash_Write(const Fls_AddressType u32LogicAddr, const Fls_LengthType u32DataLength, const uint8 * const pDataPtr)

Function	Fls_LLDReturnType Fls_Flash_Write(const Fls_AddressType u32LogicAddr, const Fls_LengthType u32DataLength, const uint8 * const pDataPtr)	
Description	Write one or more complete flash pages into given flash sector.	
Parameters	Parameter	Parameter
	u32LogicAddr	The logic address.
	u32DataLength	Data length in bytes.
	pDataPtr	Data to be written.
Returns	Fls_LLDReturnType	
Referenced By	Fls_IPW_Write	

2.8.1.8 Fls_LLDReturnType Fls_Flash_Read (const Fls_AddressType u32LogicAddr, Fls_LengthType u32DataLength, uint8 * pDataPtr)

Function	Fls_LLDReturnType Fls_Flash_Read (const Fls_AddressType u32LogicAddr, Fls_LengthType u32DataLength, uint8 * pDataPtr)	
Description	Read one or more bytes from given flash sector.	
Parameters	Parameter	Description

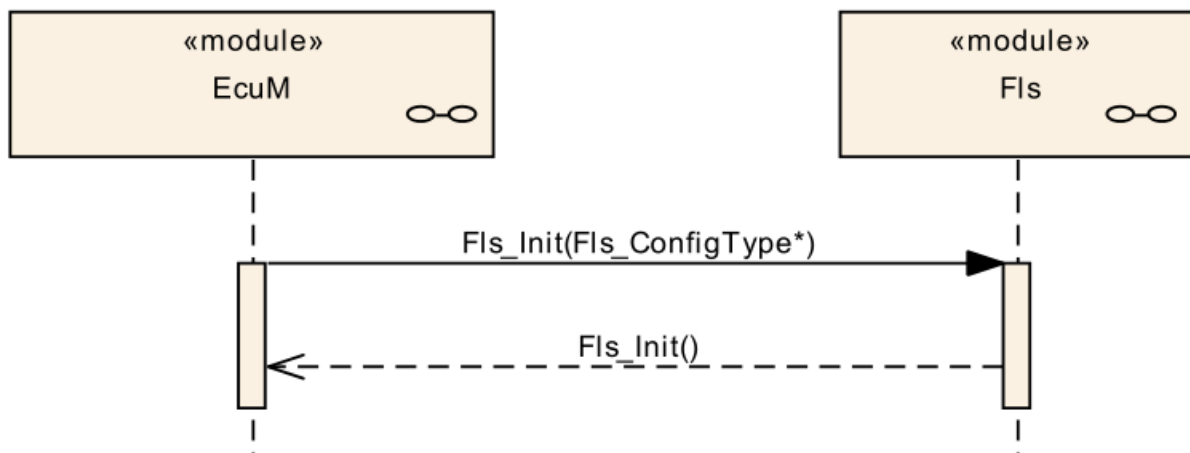
	u32LogicAddr	The logic address.
	u32DataLength	Data length in bytes.
	pDataPtr	Where read data to be written.
Returns	Fls_LLDReturnType	
Referenced By	Fls_IPW_Read	

2.8.1.9 Fls_LLDReturnType Fls_Flash_Compare (const Fls_AddressType u32LogicAddr, const Fls_LengthType u32DataLength, const uint8 * const pDataPtr)

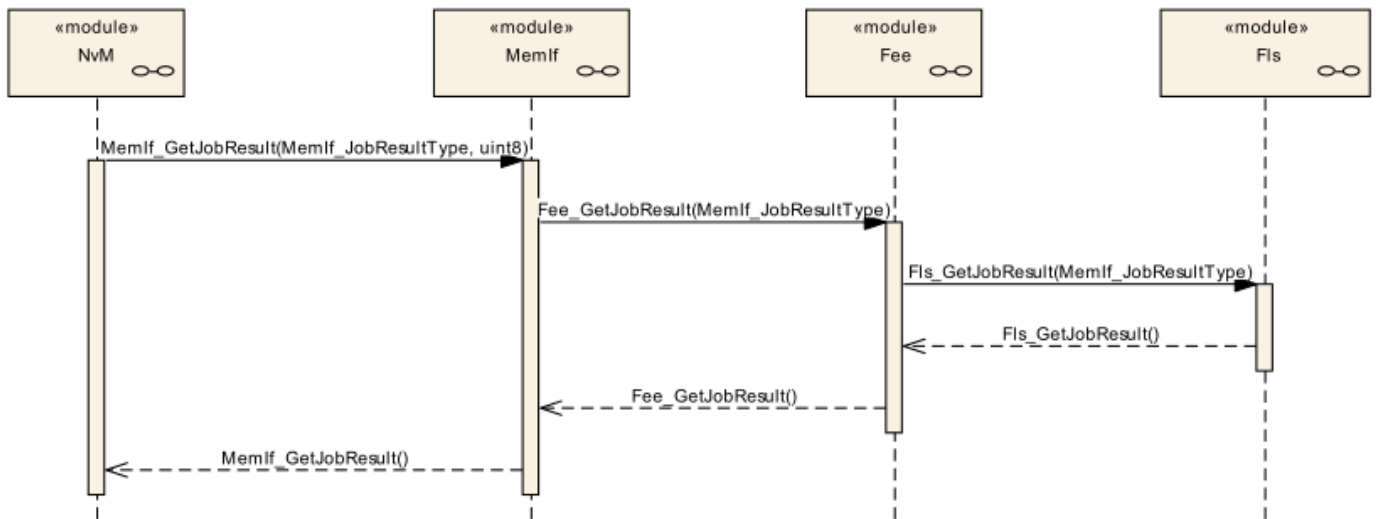
Function	Fls_LLDReturnType Fls_Flash_Compare (const Fls_AddressType u32LogicAddr, const Fls_LengthType u32DataLength, const uint8 * const pDataPtr)	
Description	Compare data buffer with content of given flash sector.	
Parameters	Parameter	Description
	u32LogicAddr	The logic address.
	u32DataLength	Data length in bytes.
	pDataPtr	Data to be compared.
Returns	Fls_LLDReturnType	
Referenced By	Fls_IPW_Compare	

2.9 API Sequence Diagram

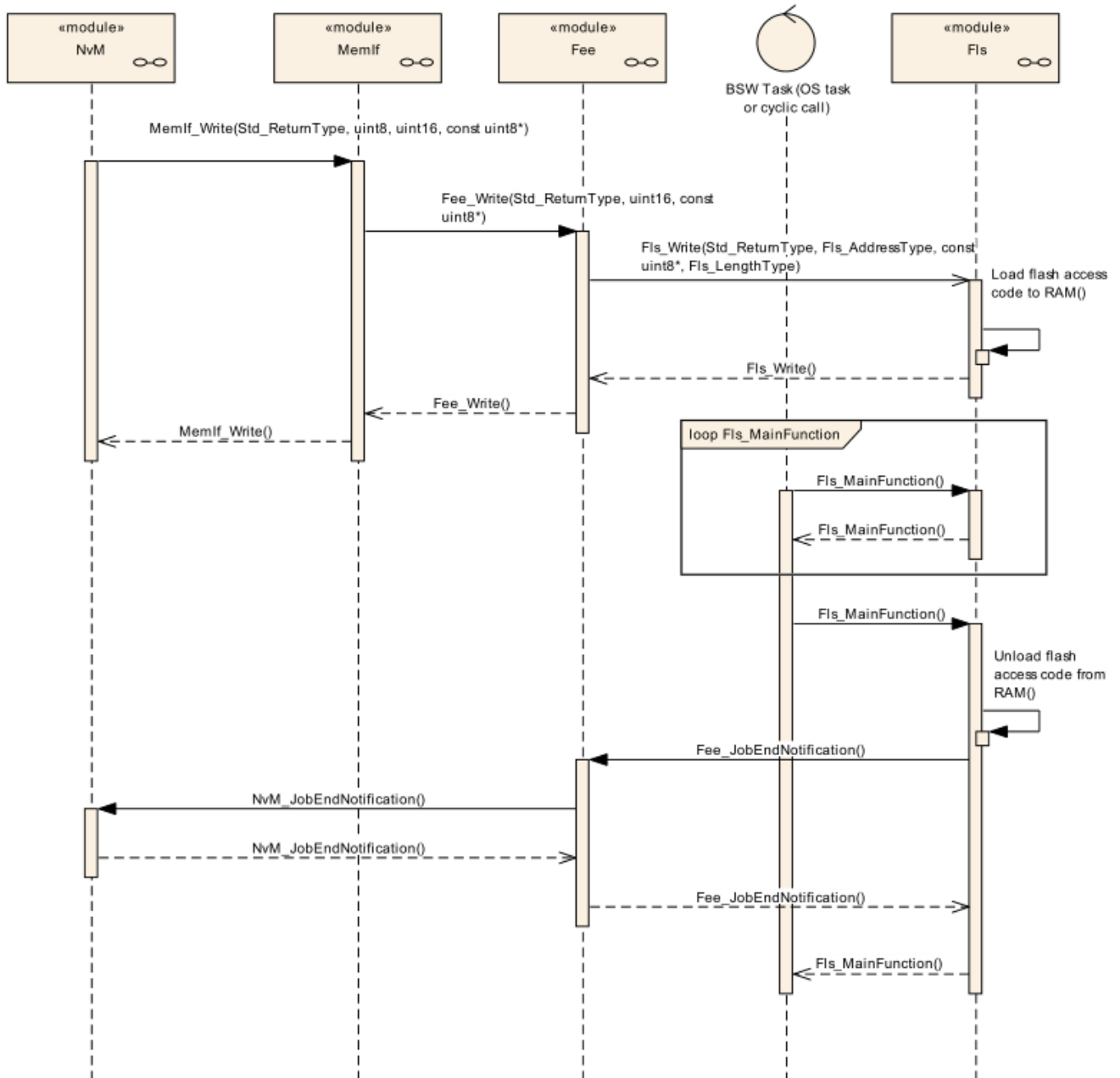
2.9.1 Initialization



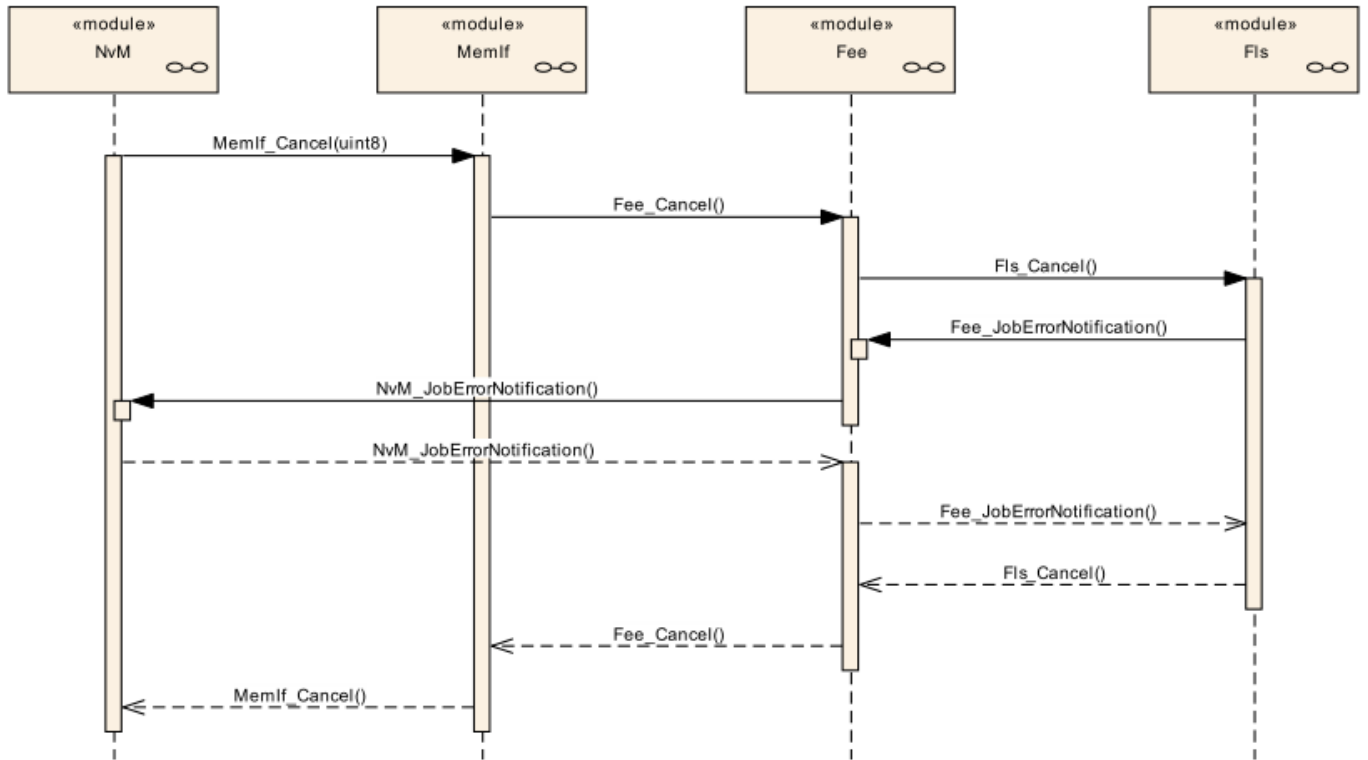
2.9.2 Synchronous Functions



2.9.3 Asynchronous Functions



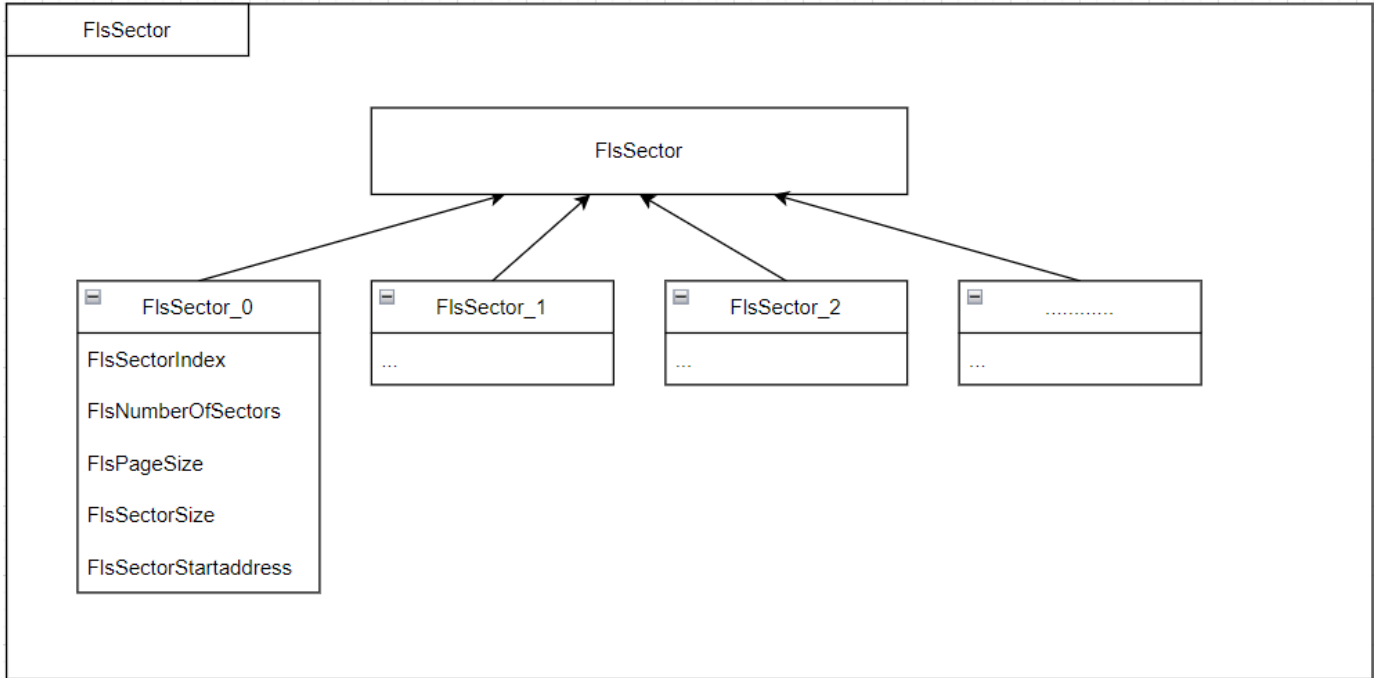
2.9.4 Canceling a Running Job



Chapter 3 Tresos Configuration Items

3.1 Container Inclusion Relation

The container inclusion relation is shown below:

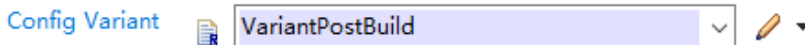


FlsGeneral	FlsConfigSet
FlsAcLoadOnJobStart	FlsAcErase
FlsBaseAddress	FlsAcWrite
FlsBlankCheckApi	FlsCallCycle
FlsCancelApi	FlsDefaultMode
FlsCompareApi	FlsJobEndNotification
FlsDevErrorDetect	FlsJobErrorNotification
FlsDriverIndex	FlsDriverIndex
FlsEraseVerificationEnabled	FlsEraseVerificationEnabled
FlsGetJobResultApi	FlsMaxReadFastMode
FlsGetStatusApi	FlsMaxReadNormalMode
FlsSetModeApi	FlsMaxWriteFastMode
FlsTotalSize	FlsMaxWriteNormalMode
FlsUseInterrupts	FlsProtection
FlsVersionInfoApi	
FlsWriteVerificationEnabled	

FlsPublishedInformation	CommonPublishedInformation
FlsAcLocationErase	ArReleaseMajorVersion
FlsAcLocationWrite	ArReleaseMinorVersion
FlsAcSizeErase	ArReleaseRevisionVersion
FlsAcSizeWrite	ModuleId
FlsEraseTime	SwMajorVersion
FlsErasedValue	SwMinorVersion
FlsExpectedHwId	SwPatchVersion
FlsSpecifiedEraseCycles	VendorId
FlsWriteTime	

3.2 Containers and Variables

3.2.1 IMPLEMENTATION_CONFIG_VARIANT

Variables	IMPLEMENTATION_CONFIG_VARIANT	
Description	N/A	
Screenshot		
Properties	Property	Value
	Type	Variable: enumeration
	Label	Config Variant
	Default	VariantPreCompile
	Range	VariantPostBuild VariantPreCompile

3.2.2 FlsGeneral

Container	FlsConfigSet	
Description	Container for general parameters of the flash driver. These parameters are always pre-compile.	
Screenshot		
Properties	Property	Value
	Type	Container

3.2.2.1 FlsAcLoadOnJobStart

Variables	FlsAcLoadOnJobStart	
Description	The flash driver shall load the flash access code to RAM whenever an erase or write job is started and unload (overwrite) it after that job has been finished or canceled.	
Screenshot		
Properties	Property	Value
	Type	Variable: Boolean

	Label	Fls Load Access Code On Job Start
	Origin	AUTOSAR_ECUC
	Default	False

3.2.2.2 FlsBaseAddress

Variables	FlsBaseAddress	
Description	The flash memory start address .	
Screenshot	Fls Base Address (0 -> 4294967295) 	
Properties	Property	Value
	Type	Variable:Integer
	Label	Fls Base Address
	Origin	AUTOSAR_ECUC
	Default	67108864

3.2.2.3 FlsBlankCheckApi

Variables	FlsBlankCheckApi	
Description	Compile switch to enable and disable the Fls_BlankCheck function.	
Screenshot	Fls Blank Check Api 	
Properties	Property	Value
	Type	Variable:Boolean
	Label	Fls Blank Check Api
	Origin	AUTOSAR_ECUC
	Default	false

3.2.2.4 FlsCancelApi

Variables	FlsCancelApi	
Description	Compile switch to enable and disable the Fls_Cancel function.	
Screenshot	Fls Cancel Api 	
Properties	Property	Value
	Type	Variable:Boolean
	Label	Fls Cancel Api
	Origin	AUTOSAR_ECUC
	Default	false

3.2.2.5 FlsCompareApi

Variables	FlsCompareApi	
Description	Compile switch to enable and disable the Fls_Compare function.	
Screenshot	Fls Compare Api 	
Properties	Property	Value
	Type	Variable:Boolean
	Label	Fls Compare Api

	Origin	AUTOSAR_ECUC
	Default	false

3.2.2.6 FlsDevErrorDetect

Variables	FlsCompareApi	
Description	Switches the Default Error Tracer (Det) detection and notification ON or OFF.	
Screenshot	Fls Development Error Detect    	
Properties	Property	Value
	Type	Variable:Boolean
	Label	Fls Development Error Detect
	Origin	AUTOSAR_ECUC
	Default	false

3.2.2.7 FlsDriverIndex

Variables	FlsDriverIndex	
Description	Index of the driver, used by FEE.	
Screenshot	Fls Driver Index (0 -> 254)  0  	
Properties	Property	Value
	Type	Variable:Integer
	Label	Fls Driver Index
	Origin	AUTOSAR_ECUC
	Default	0

3.2.2.8 FlsEraseVerificationEnabled

Variables	FlsEraseVerificationEnabled	
Description	Compile switch to enable erase verification.	
Screenshot	Fls Erase Verification  ☐  	
Properties	Property	Value
	Type	Variable:Boolean
	Label	Fls Erase Verification
	Origin	AUTOSAR_ECUC
	Default	false

3.2.2.9 FlsGetJobResultApi

Variables	FlsGetJobResultApi	
Description	Compile switch to enable and disable the Fls_GetJobResult function.	
Screenshot	Fls Get Job Result Api    	
Properties	Property	Value
	Type	Variable:Boolean
	Label	Fls Get Job Result Api
	Origin	AUTOSAR_ECUC

	Default	false
--	---------	-------

3.2.2.10 FlsGetStatusApi

Variables	FlsGetStatusApi	
Description	Compile switch to enable and disable the Fls_GetStatus function.	
Screenshot	Fls Get Status Api    	
Properties	Property	Value
	Type	Variable:Boolean
	Label	Fls Get Status Api
	Origin	AUTOSAR_ECUC
	Default	false

3.2.2.11 FlsSetModeApi

Variables	FlsGetJobResultApi	
Description	Compile switch to enable and disable the Fls_SetMode function.	
Screenshot	Fls Set Mode Api    	
Properties	Property	Value
	Type	Variable:Boolean
	Label	Fls Set Mode Api
	Origin	AUTOSAR_ECUC
	Default	false

3.2.2.12 FlsTimeoutSupervisionEnabled

Variables	FlsTimeoutSupervisionEnabled	
Description	Compile switch to enable timeout supervision.	
Screenshot	Fls Timeout Supervision    	
Properties	Property	Value
	Type	Variable:Boolean
	Label	Fls Timeout Supervision
	Origin	AUTOSAR_ECUC
	Default	false

3.2.2.13 FlsTotalSize

Variables	FlsTotalSize	
Description	The total amount of flash memory in bytes.	
Screenshot	Fls Total Size (0 -> 4294967295)  	
Properties	Property	Value
	Type	Variable:Integer
	Label	Fls Total Size
	Origin	AUTOSAR_ECUC
	Default	0

3.2.2.14 FlsUseInterrupts

Variables	FlsUseInterrupts	
Description	Job processing triggered by hardware interrupt.	
Screenshot		
Properties	Property	Value
	Type	Variable:Boolean
	Label	Fls Use Interrupts
	Origin	AUTOSAR_ECUC
	Default	false

3.2.2.15 FlsVersionInfoApi

Variables	FlsVersionInfoApi	
Description	Pre-processor switch to enable / disable the API to read out the modules version information.	
Screenshot		
Properties	Property	Value
	Type	Variable:Boolean
	Label	Fls Version Info Api
	Origin	AUTOSAR_ECUC
	Default	false

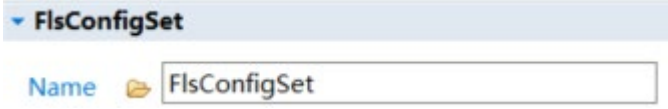
3.2.2.16 FlsWriteVerificationEnabled

Variables	FlsWriteVerificationEnabled	
Description	Compile switch to enable write verification.	
Screenshot		
Properties	Property	Value
	Type	Variable:Boolean
	Label	Fls Write Verification
	Origin	AUTOSAR_ECUC
	Default	false

3.2.2.17 FlsEcucPartitionRef

Variables	FlsEcucPartitionRef	
Description	Maps the Flash driver to zero or one ECUC partition to make the driver API available in this partition. Not used in driver code.	
Screenshot		
Properties	Property	Value
	Type	Container
	Label	FlsEcucPartitionRef
	Origin	AUTOSAR_ECUC
	Default	N/A

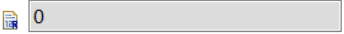
3.2.3 FlsConfigSet

Container	FlsConfigSet	
Description	Container for general parameters of the flash driver. These parameters are always pre-compile.	
Screenshot		
Properties	Property	Value
	Type	Container

3.2.3.1 FlsAcErase

Variables	FlsAcErase	
Description	Address offset in RAM to which the erase flash access code shall be loaded. Used as function pointer to access the erase flash access code.	
Screenshot		
Properties	Property	Value
	Type	Variable: Integer
	Label	Fls Access Code Erase
	Origin	AUTOSAR_ECUC
	Default	0

3.2.3.2 FlsAcWrite

Variables	FlsAcWrite	
Description	Address offset in RAM to which the write flash access code shall be loaded. Used as function pointer to access the write flash access code.	
Screenshot		
Properties	Property	Value
	Type	Variable: Integer
	Label	Fls Access Code Write
	Origin	AUTOSAR_ECUC
	Default	0

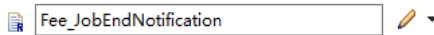
3.2.3.3 FlsCallCycle

Variables	FlsCallCycle	
Description	Cycle time of calls of the flash driver main function	
Screenshot		
Properties	Property	Value
	Type	Variable:Float
	Label	Fls Call Cycle
	Origin	AUTOSAR_ECUC
	Default	0.0

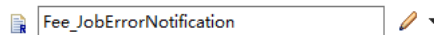
3.2.3.4 FlsDefaultMode

Variables	FlsDefaultMode	
Description	This parameter is the default FLS device mode after initialization.	
Screenshot	 	
Properties	Property	Value
	Type	Variable: enumeration
	Label	Fls Default Mode
	Origin	AUTOSAR_ECUC
	Default	MEMIF_MODE_SLOW

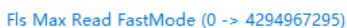
3.2.3.5 FlsJobEndNotification

Variables	FlsJobEndNotification	
Description	Mapped to the job end notification routine provided by some upper layer module, typically the Fee module.	
Screenshot	 	
Properties	Property	Value
	Type	Variable: String
	Label	Fls Job End Notification
	Origin	AUTOSAR_ECUC
	Default	Fee_JobEndNotification

3.2.3.6 FlsJobErrorNotification

Variables	FlsJobErrorNotification	
Description	Mapped to the job error notification routine provided by some upper layer module, typically the Fee module.	
Screenshot	 	
Properties	Property	Value
	Type	Variable: String
	Label	Fls Job Error Notification
	Origin	AUTOSAR_ECUC
	Default	Fee_JobErrorNotification

3.2.3.7 FlsMaxReadFastMode

Variables	FlsMaxReadFastMode	
Description	The maximum number of bytes to read or compare in one cycle of the flash driver's job processing function in fast mode.	
Screenshot	 	
Properties	Property	Value
	Type	Variable: Integer
	Label	Fls Max Read FastMode
	Origin	AUTOSAR_ECUC
	Default	1048576

3.2.3.8 FlsMaxReadNormalMode

Variables	FlsMaxReadNormalMode	
Description	The maximum number of bytes to read or compare in one cycle of the flash driver's job processing function in normal mode.	
Screenshot	Fls Max Read Normal Mode (0 -> 4294967295) 	
Properties	Property	Value
	Type	Variable: Integer
	Label	Fls Max Read Normal Mode
	Origin	AUTOSAR_ECUC
	Default	1024

3.2.3.9 FlsMaxWriteFastMode

Variables	FlsMaxWriteFastMode	
Description	The maximum number of bytes to write in one cycle of the flash driver's job processing function in fast mode.	
Screenshot	Fls Max Write Fast Mode 	
Properties	Property	Value
	Type	Variable: Integer
	Label	Fls Max Write Fast Mode
	Origin	AUTOSAR_ECUC
	Default	128

3.2.3.10 FlsMaxWriteNormalMode

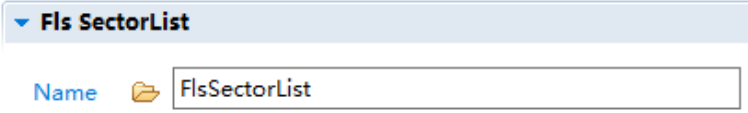
Variables	FlsMaxWriteNormalMode	
Description	The maximum number of bytes to write in one cycle of the flash driver's job processing function in normal mode.	
Screenshot	Fls Max Write Normal Mode 	
Properties	Property	Value
	Type	Variable: Integer
	Label	Fls Max Write Normal Mode
	Origin	AUTOSAR_ECUC
	Default	16

3.2.3.11 FlsProtection

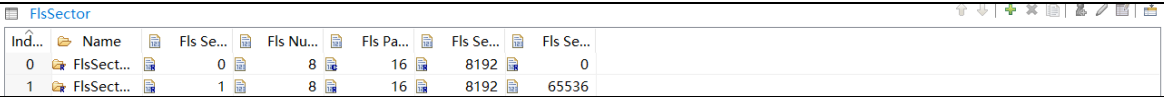
Variables	FlsMaxWriteNormalMode	
Description	Erase/write protection settings. Only relevant if supported by hardware.	
Screenshot	Fls Protection (0 -> 4294967295) 	
Properties	Property	Value
	Type	Variable: Integer
	Label	Fls Protection

	Origin	AUTOSAR_ECUC
	Default	1

3.2.4 FlsSectorList

Container	FlsSectorList	
Description	List of flashable sectors and pages.	
Screenshot		
Properties	Property	Value
	Type	Container

3.2.4.1 FlsSector

List	FlsSector	
Description	Erase/write protection settings. Only relevant if supported by hardware.	
Screenshot		
Properties	Property	Value
	Type	list
	Label	FlsSector

3.2.4.1.1 Fls Sector Index

Variable	Fls Sector Index	
Description	Fls Sector Index is an invariant index, used to order flash sectors and loop over them in the correct, configured order.	
Screenshot		
Properties	Property	Value
	Type	Variable:Integer
	Label	Fls Sector Index
	Origin	FLAGCHIP
	Editable	False

3.2.4.1.2 FlsNumberOfSectors

Variable	FlsNumberOfSectors	
Description	The number of specified PortPins in this PortContainer.	
Screenshot		
Properties	Property	Value
	Type	Variable: Integer
	Label	Fls Number Of Sector
	Origin	AUTOSAR_ECUC
	Editable	True
	Default	1

3.2.4.1.3 FlsPageSize

List	FlsPageSize	
Description	Size of one page of this sector.	
Screenshot	Fls Page Size (0 -> 4294967295) 	
Properties	Property	Value
	Type	Variable: Integer
	Label	Fls Page Size
	Origin	AUTOSAR_ECUC
	Editable	False
	Default	16

3.2.4.1.4 FlsSectorSize

Container	FlsSectorSize	
Description	Size of this sector.	
Screenshot	Fls Sector Size (0 -> 4294967295) 	
Properties	Property	Value
	Type	Container
	Label	Fls Sector Size
	Origin	AUTOSAR_ECUC
	Editable	False
	Default	8192

3.2.4.1.5 FlsSectorStartaddress

Variable	FlsSectorStartaddress	
Description	Start address of this sector.	
Screenshot	Fls Sector Start Address (0 -> 4294967295) 	
Properties	Property	Value
	Type	Variable: Integer
	Label	Fls Sector Start Address
	Editable	True
	Default	0

3.2.5 FlsPublishedInformation

Container	FlsPublishedInformation	
Description	Additional published parameters not covered by CommonPublishedInformation container.	
Screenshot	Name  FlsPublishedInformation	
Properties	Property	Value
	Type	Container

3.2.5.1 FlsAcLocationErase

Variable	FlsAcLocationErase	
Description	Position in RAM, to which the erase flash access code has to be loaded.	
Screenshot		
Properties	Property	Value
	Type	Variable: Integer
	Label	Fls Access Code Location Erase
	Origin	AUTOSAR_ECUC
	Default	0

3.2.5.2 FlsAcLocationWrite

Variable	FlsAcLocationWrite	
Description	Position in RAM, to which the write flash access code has to be loaded.	
Screenshot		
Properties	Property	Value
	Type	Variable: Integer
	Label	Fls Access Code Location Write
	Origin	AUTOSAR_ECUC
	Default	0

3.2.5.2.1 FlsAcSizeErase

Variable	FlsAcSizeErase	
Description	Number of bytes in RAM needed for the erase flash access code.	
Screenshot		
Properties	Property	Value
	Type	Variable: Integer
	Label	Fls Access Code Size Erase
	Origin	AUTOSAR_ECUC
	Default	0

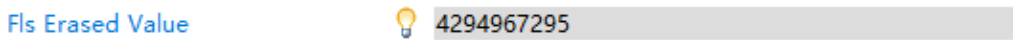
3.2.5.3 FlsAcSizeWrite

Variable	FlsAcSizeWrite	
Description	Number of bytes in RAM needed for the write flash access code.	
Screenshot		
Properties	Property	Value
	Type	Variable: Integer
	Label	Fls Access Code Size Write
	Origin	AUTOSAR_ECUC
	Default	0

3.2.5.4 FlsEraseTime

Variable	FlsEraseTime	
Description	Maximum time to erase one complete flash sector [sec].	
Screenshot		
Properties	Property	Value
	Type	Variable: Float
	Label	Fls Erase Time
	Origin	AUTOSAR_ECUC
	Default	0.0032

3.2.5.5 FlsErasedValue

Variable	FlsErasedValue	
Description	The contents of an erased flash memory cell.	
Screenshot		
Properties	Property	Value
	Type	Fls Erased Value
	Label	Port VersionInfo Api
	Origin	AUTOSAR_ECUC
	Default	4294967295

3.2.5.6 FlsExpectedHwId

Variable	FlsExpectedHwId	
Description	Unique identifier of the hardware device that is expected by this driver.	
Screenshot		
Properties	Property	Value
	Type	Variable: String
	Label	Fls Expected Hw Id
	Origin	AUTOSAR_ECUC
	Default	0

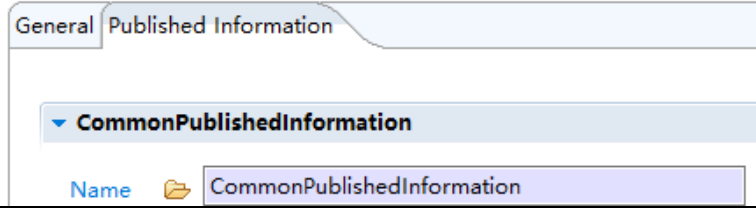
3.2.5.7 FlsSpecifiedEraseCycles

Variable	FlsSpecifiedEraseCycles	
Description	Number of erase cycles specified for the flash device.	
Screenshot		
Properties	Property	Value
	Type	Variable: Integer
	Label	Fls Specified Erase Cycles
	Origin	AUTOSAR_ECUC
	Default	100000

3.2.5.8 FlsWriteTime

Variable	FlsWriteTime	
Description	Maximum time to program one complete flash page [sec].	
Screenshot		
Properties	Property	Value
	Type	Variable: Float
	Label	Fls Write Time
	Origin	AUTOSAR_ECUC
	Default	1.2E-4

3.2.6 CommonPublishedInformation

Container	CommonPublishedInformation	
Description	Common container, aggregated by all modules. It contains published information about vendor and versions.	
Screenshot		
Properties	N/A	

3.2.6.1 ArReleaseMajorVersion

Variable	ArReleaseMajorVersion	
Description	Major version number of AUTOSAR specification on which the appropriate implementation is based.	
Screenshot		
Properties	Property	Value
	Type	Variable: Integer_Label
	Origin	FLAGCHIP
	SymbolicNameValue	false
	Default	4

3.2.6.2 ArReleaseMinorVersion

Variable	ArReleaseMinorVersion	
Description	Minor version number of AUTOSAR specification on which the appropriate implementation is based.	
Screenshot		
Properties	Property	Value
	Type	Variable: Integer_Label
	Origin	FLAGCHIP
	SymbolicNameValue	False
	Default	6

3.2.6.3 ArReleaseRevisionVersion

Variable	ArReleaseRevisionVersion	
Description	Revision version number of AUTOSAR specification on which the appropriate implementation is based.	
Screenshot		
Properties	Property	Value
	Type	Variable: Integer_Label
	Origin	FLAGCHIP
	SymbolicNameValue	False
	Default	0

3.2.6.4 SwMajorVersion

Variable	SwMajorVersion	
Description	Major version number of the vendor specific implementation of the module. The numbering is vendor specific.	
Screenshot		
Properties	Property	Value
	Type	Variable: Integer_Label
	Origin	FLAGCHIP
	SymbolicNameValue	False
	Default	0

3.2.6.5 SwMinorVersion

Variable	SwMinorVersion	
Description	Minor version number of the vendor specific implementation of the module. The numbering is vendor specific.	
Screenshot		
Properties	Property	Value
	Type	Variable: Integer_Label
	Origin	FLAGCHIP
	SymbolicNameValue	False
	Default	3

3.2.6.6 SwPatchVersion

Variable	SwPatchVersion	
Description	Patch level version number of the vendor specific implementation of the module. The numbering is vendor specific.	
Screenshot		
Properties	Property	Value
	Type	Variable: Integer_Label
	Origin	FLAGCHIP
	SymbolicNameValue	False
	Default	0

3.2.6.7 ModuleId

Variable	ModuleId	
Description	Module ID of this module from Module List.	
Screenshot		
Properties	Property	Value
	Type	Variable: Integer_Label
	Origin	FLAGCHIP
	SymbolicNameValue	False
	Default	92

3.2.6.8 VendorId

Variable	VendorId	
Description	Vendor ID of the dedicated implementation of this module according to the AUTOSAR vendor list.	
Screenshot		
Properties	Property	Value
	Type	Variable: Integer_Label
	Origin	FLAGCHIP
	SymbolicNameValue	False
	Default	174

Chapter 4 Configuration Guides

4.1 Configuration Item Constraint

- 1) In the FlsGeneral container, the Fls Base Address will match the response value according to the CommonSubdivisive option of the Common module. If the value does not match, an error will be reported. You need to click Calculate Value to match again.

The screenshot shows the 'Fls General' configuration window. The 'Name' field is 'FlsGeneral'. The 'Fls Base Address' field is set to '26214' and is highlighted with a red box. To the right of this field is a 'Calculate Value' button, also highlighted with a red box. Other fields include 'Fls Load Access Code On Job Start', 'Fls Blank Check Api', 'Fls Compare Api', 'Fls Driver Index (0 -> 254)', and 'Fls Erase Verification'.

- 2) If the Fee module is used, in the FlsConfigSet container, the FlsMaxWriteNormalMode must be no less than the FeeVirtualPageSize in the FeeGeneral container in the Fee module, or an error will be reported.

The screenshot shows the 'Fee' configuration window with the 'Fee General' tab selected. The 'Name' field is 'FeeGeneral'. The 'Fee Virtual Page Size' field is set to '32' and is highlighted with a red box. Other fields include 'Fee Development Error Detect', 'FeeMainFunctionPeriod (0.0000001 -> 100000)', 'Fee Nvm Job End Notification', 'Fee Nvm Job Error Notification', 'Fee Polling Mode', 'Fee Version Info Api', and 'Fee Data Buffer Size'.

FLS

Name FLS

General FLSector Published Information

FLSConfigSet

Name FLSConfigSet

FLS Access Code Erase (0 -> 4294967295)	0
FLS Access Code Write (0 -> 4294967295)	0
FLS Call Cycle (0 -> 1)	0.0
FLS Default Mode	MEMIF_MODE_SLOW
FLS Job End Notification	Fee_JobEndNotification
FLS Job Error Notification	Fee_JobErrorNotification
FLS Max Read FastMode (0 -> 4294967295)	1048576
FLS Max Read Normal Mode (0 -> 4294967295)	1024
FLS Max Write Fast Mode	120
FLS Max Write Normal Mode*	16
FLS Protection (0 -> 4294967295)	0

4.2 FLS Usage Common Steps

Basically, the FLS module can be configured by following the below 4 steps:

- 1) Choose which API will be used.
- 2) Configure response options or parameters according to flash performance and requirements
- 3) Generate configuration files.

4.3 Demo Configuration (FC7300F8M)

FLS General

Name FLSGeneral

FLS Load Access Code On Job Start	☐
FLS Base Address	67108864
FLS Blank Check Api	☐ FLS Cancel Api ☒
FLS Compare Api	☐ FLS Development Error Detect ☒
FLS Driver Index (0 -> 254)	0
FLS Erase Verification	☐ FLS Get Job Result Api ☒
FLS Get Status Api	☒ FLS Set Mode Api ☒
FLS Timeout Supervision	☐
FLS Total Size (0 -> 4294967295)	0
FLS Use Interrupts	☐ FLS Version Info Api ☐
FLS Write Verification	☐

FlsConfigSet

Name	Value
Fls Access Code Erase (0 -> 4294967295)	0
Fls Access Code Write (0 -> 4294967295)	0
Fls Call Cycle (0 -> 1)	0.0
Fls Default Mode	MEMIF_MODE_SLOW
Fls Job End Notification	Fee_JobEndNotification
Fls Job Error Notification	Fee_JobErrorNotification
Fls Max Read FastMode (0 -> 4294967295)	1048576
Fls Max Read Normal Mode (0 -> 4294967295)	1024
Fls Max Write Fast Mode	128
Fls Max Write Normal Mode	16
Fls Protection (0 -> 4294967295)	1

FlsSector

Name	Value
FlsSector_0	
Fls Sector	
Fls Sector Index	0
Fls Number Of Sector (0 -> 65535)	8
Fls Page Size	16
Fls Sector Size	8192
Fls Sector Start Address (0 -> 4294967295)	0

FlsSector

Name	Value
FlsSector_1	
Fls Sector	
Fls Sector Index	1
Fls Number Of Sector (0 -> 65535)	8
Fls Page Size	16
Fls Sector Size	8192
Fls Sector Start Address (0 -> 4294967295)	65536